Code repair with evidence

A working application for supported repairs, with results a developer can inspect.

Real code execution.
Visible tests.
Developer review.



versionbridge.pages.dev

Feature walkthrough, system architecture and fresh demo captures
Prepared 9 November 2026

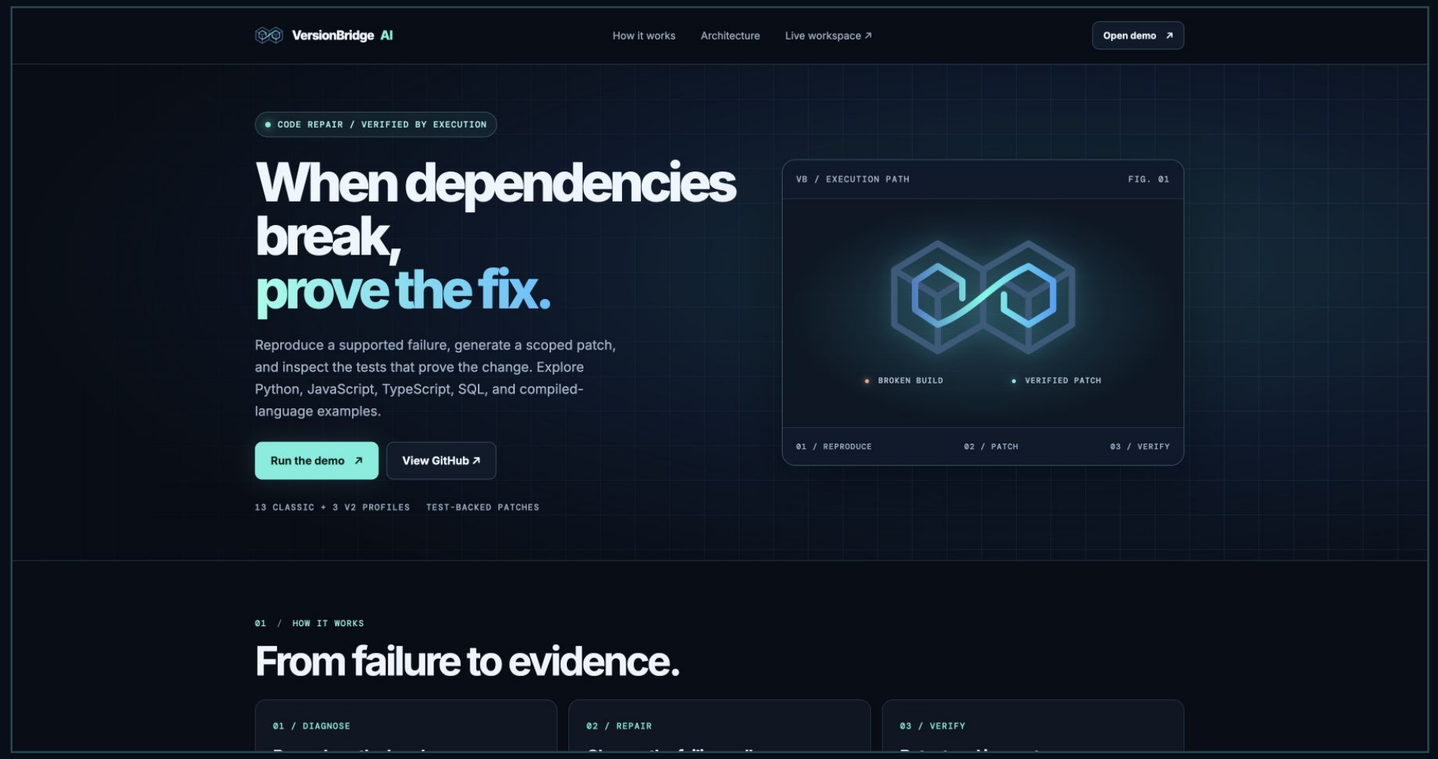
The developer problem

A library update can break an app that worked before.

01 An import or function disappears after an upgrade.

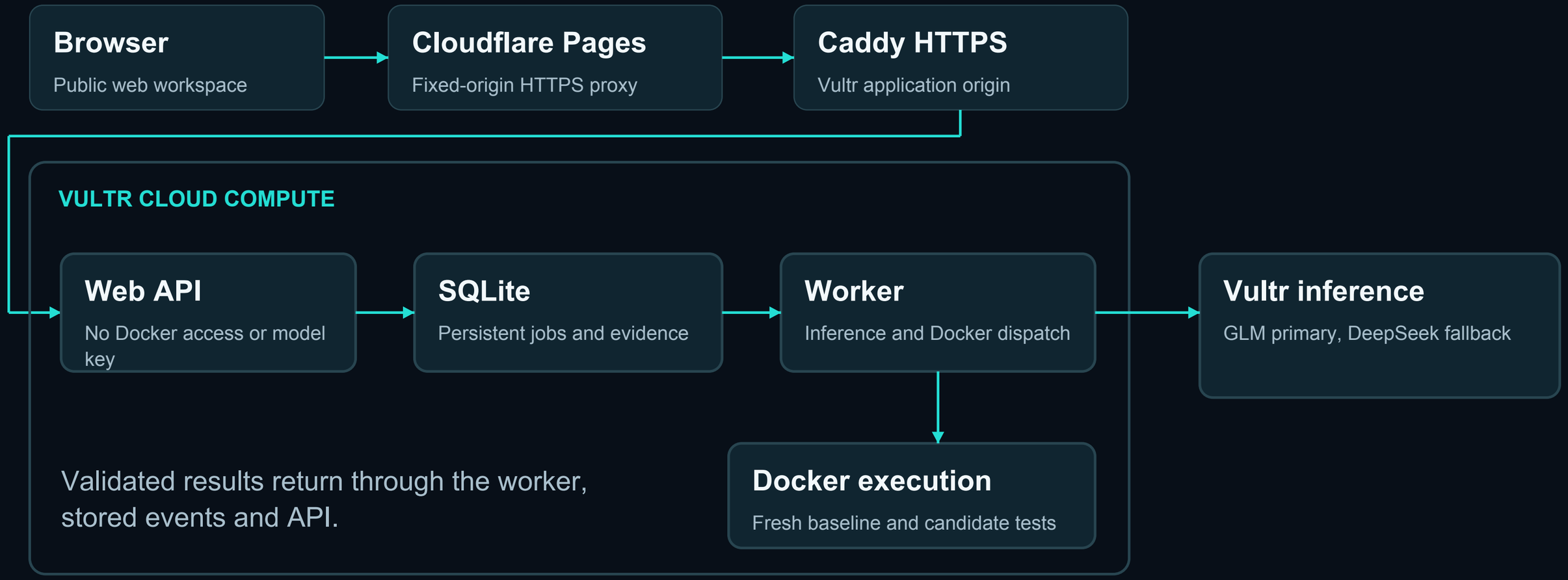
02 An AI suggestion still needs a test.

03 A change that fixes one error can break another behavior.



Deployment architecture

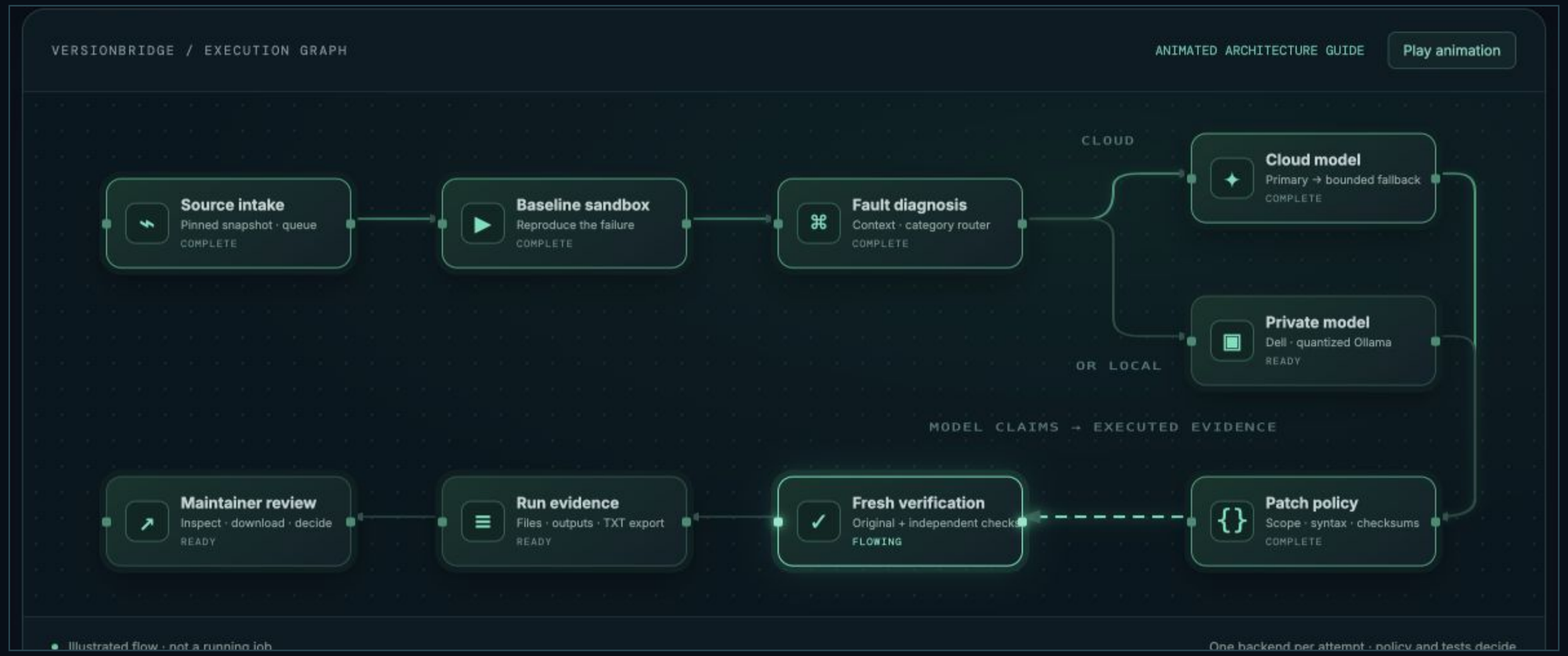
Vultr runs the application control layer and real execution worker.



Cloudflare supplies the clean address. The web app, queue and worker run on Vultr.

The architecture on the homepage

This is an interactive explanatory guide, labelled separately from a live job.



Select a node for its role. Pause or resume the guide animation. The private model is an alternative backend.

Supported repair profiles

16 reviewed profiles across 9 runtime environments. Support is bounded to these examples.

Python	3 model-authored V2 migrations; pandas, settings, arithmetic, search and regression examples
JavaScript	Discount cap example with Node execution
TypeScript	Inventory field repair with strict compilation and runtime checks
Go / Rust	Pagination and inclusive-boundary examples
Java / C++	Role comparison and integer-overflow examples
SQLite / PostgreSQL	Query repairs against disposable fixture databases

Python V2 permits scoped model-authored edits. Other examples use narrower reviewed policies.

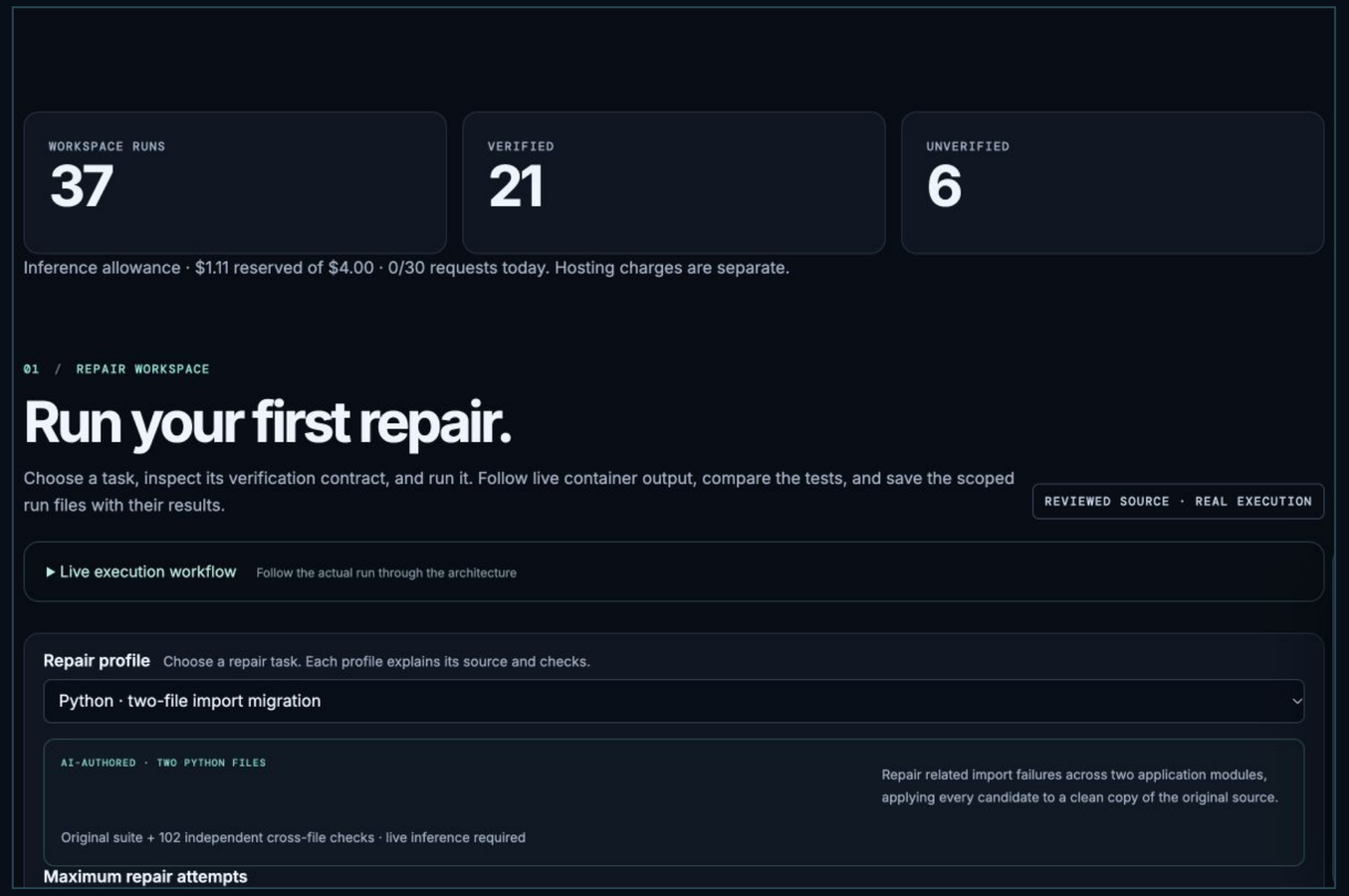
Starting a real repair

Fresh public run: Python two-file import migration, one bounded model attempt.

01 Read the profile source and verification contract.

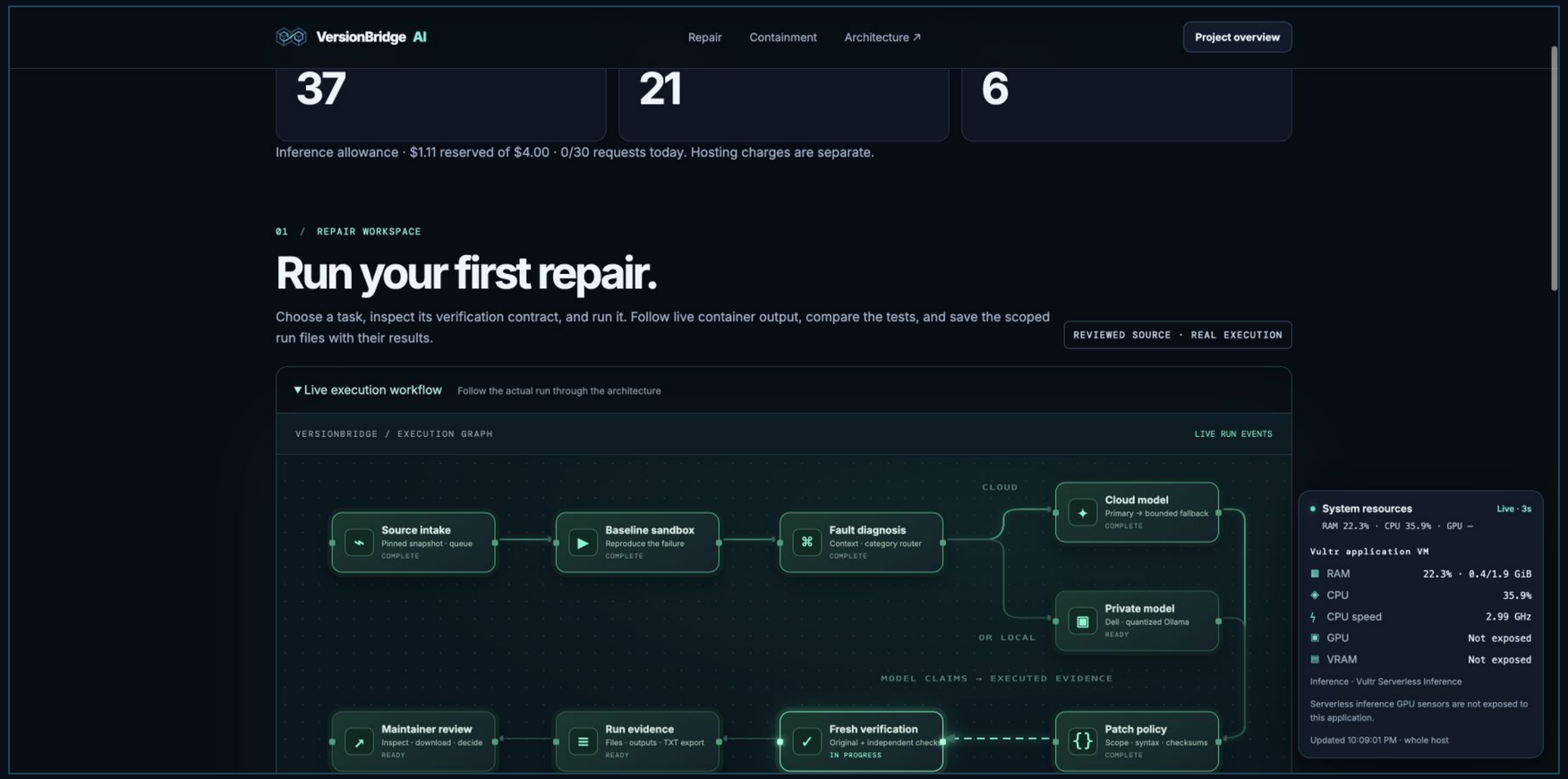
02 Choose the attempt limit and select Run repair.

03 The baseline must reproduce a failure before repair verification.



The workflow follows real events

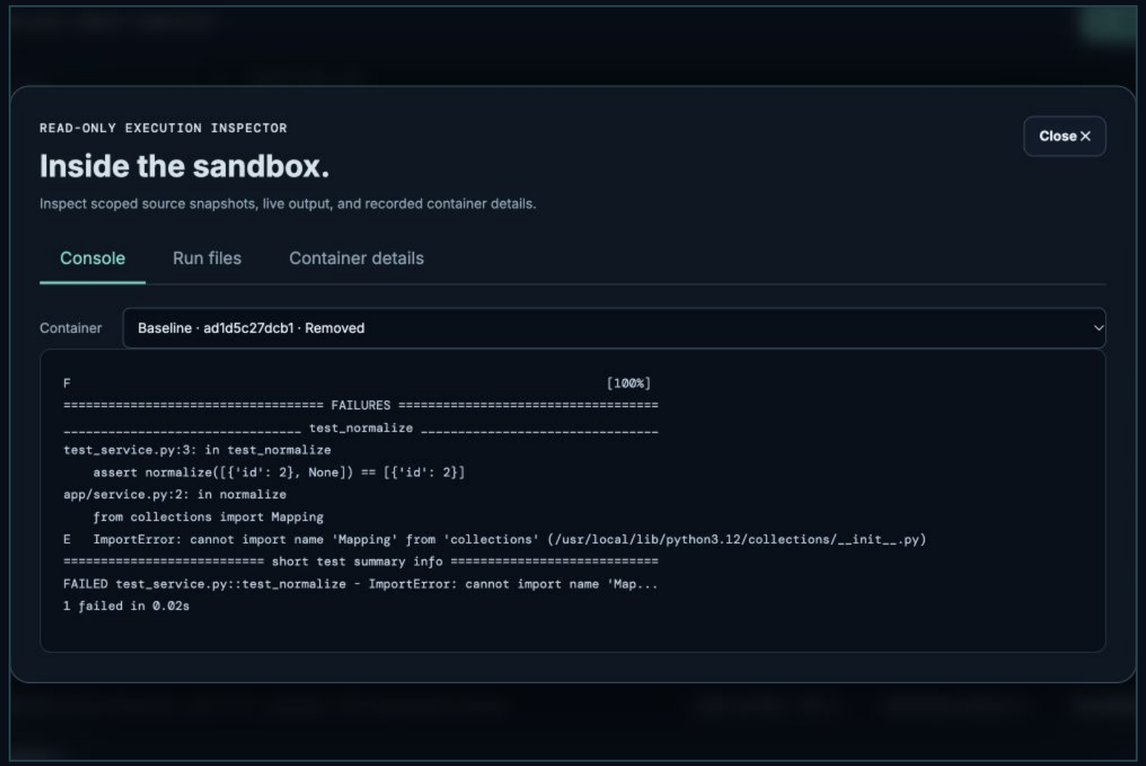
The fresh job progressed through cloud inference, patch policy and verification.



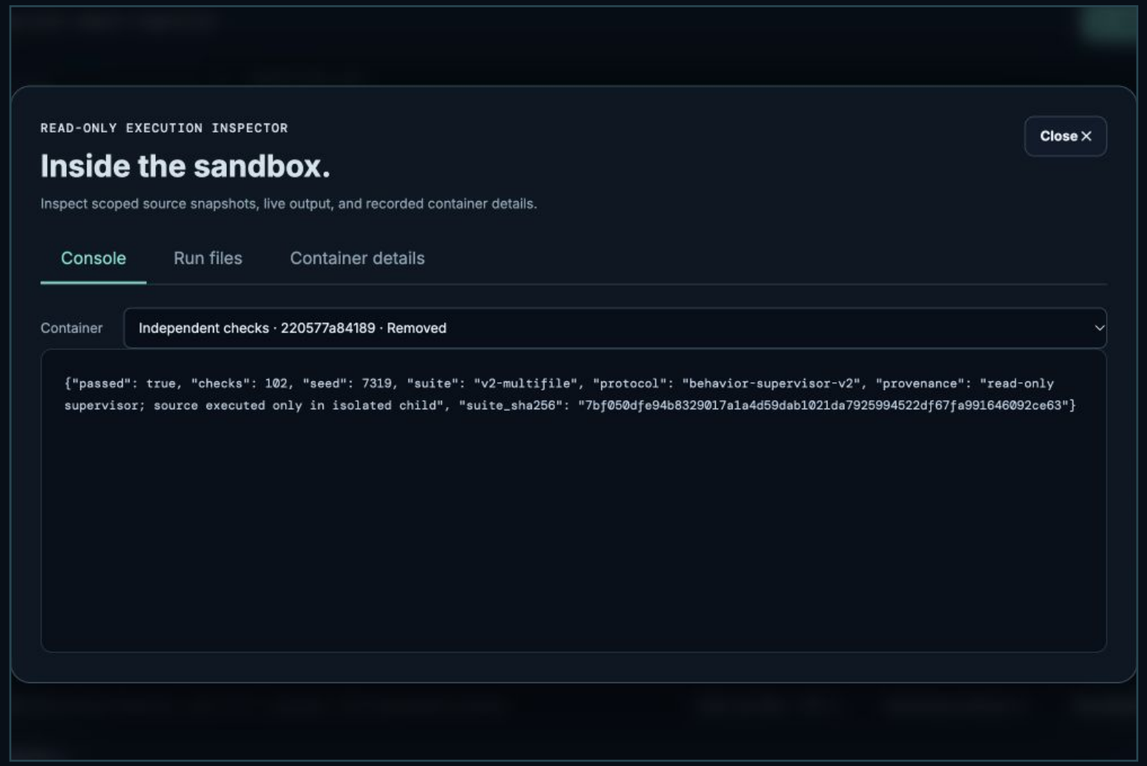
Captured while the UI reported RUNNING. Cloud inference completed; verification was in progress.

Baseline failure and independent checks

Separate fresh containers reproduce the failure and check candidate behavior.



BEFORE: ImportError, exit 1

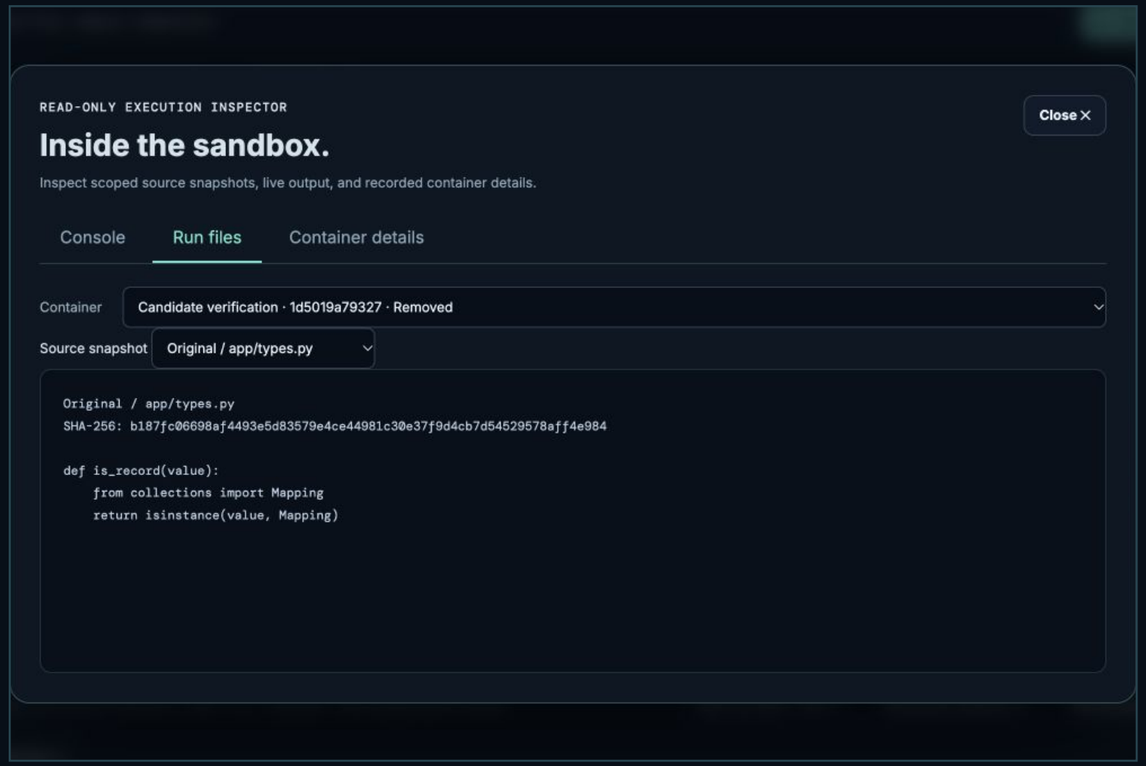


AFTER: 102 independent checks passed

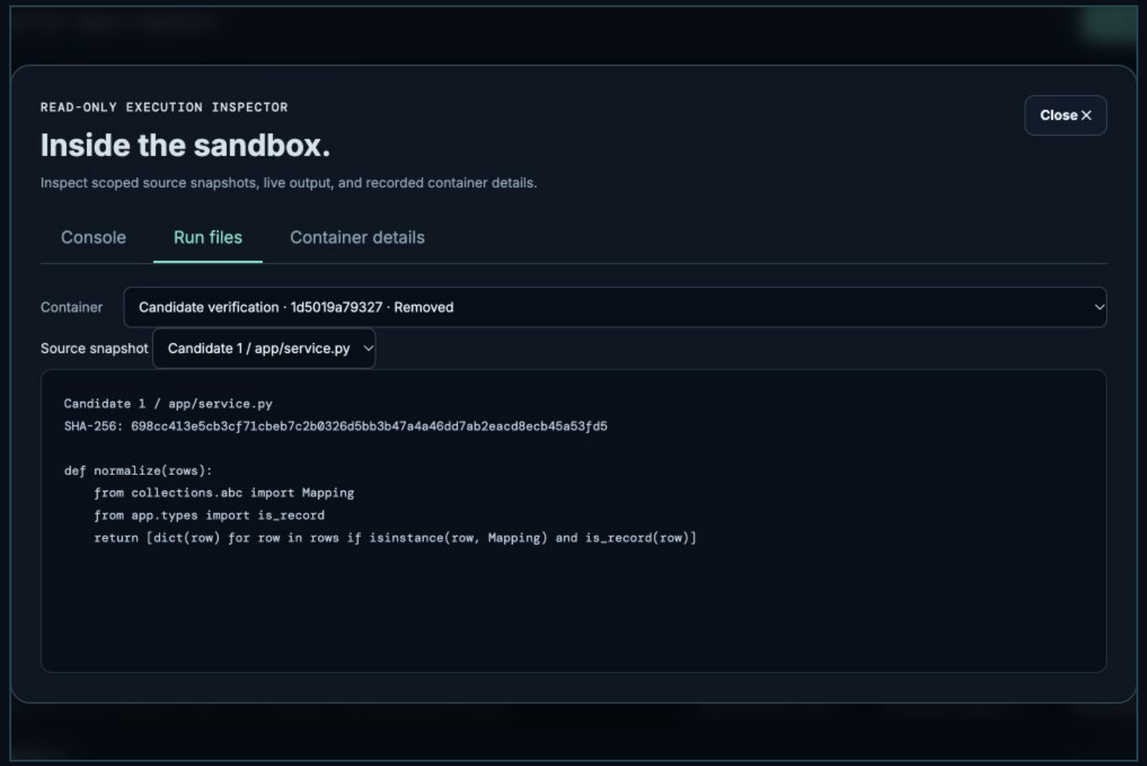
Live app captures: 8 Nov 2026

Original and candidate source files

The inspector shows actual scoped source snapshots with SHA-256 hashes.



Original: from collections import Mapping



Candidate: from collections.abc import Mapping

Container details and execution limits

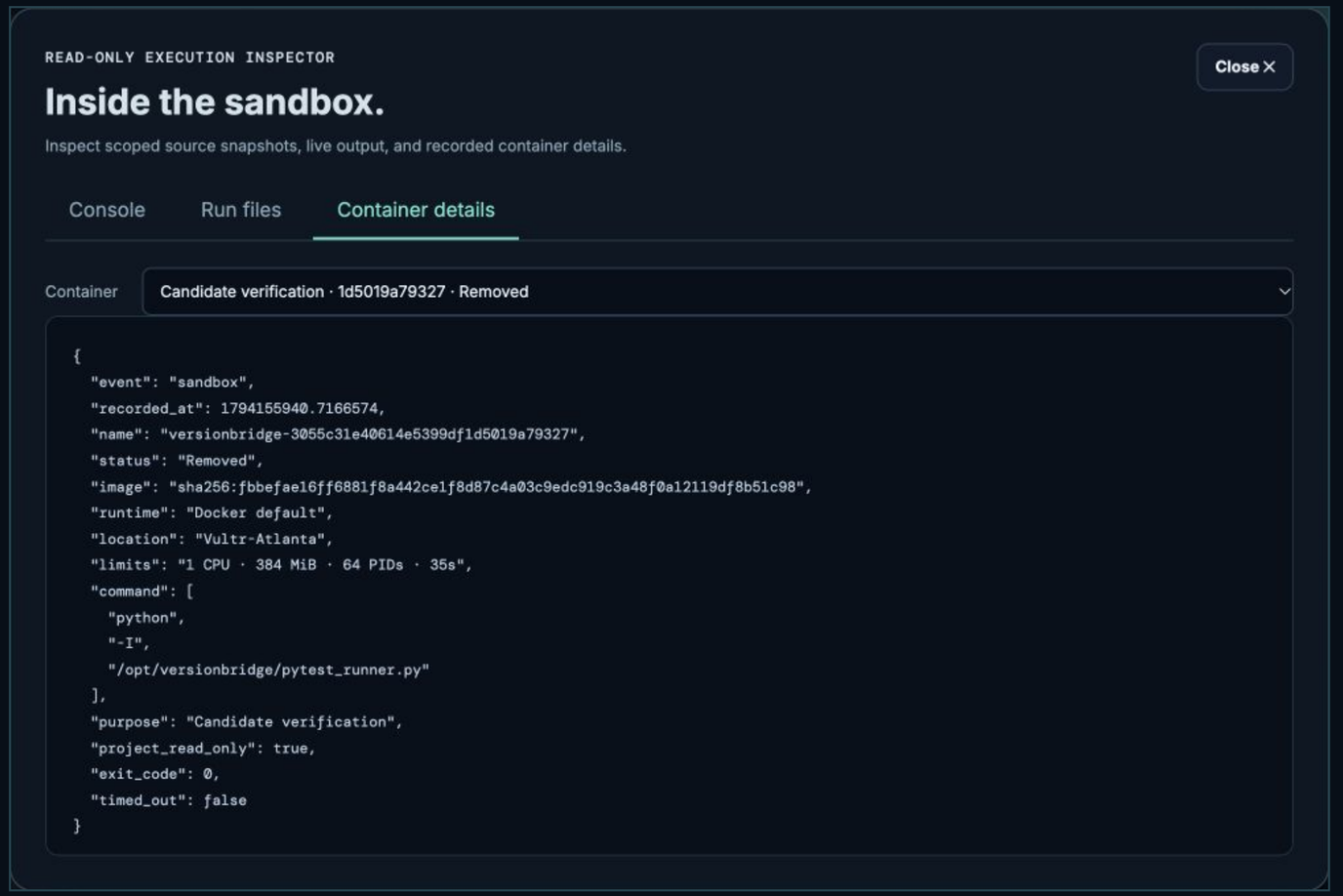
The read-only inspector exposes the image, command, location and cleanup result.

01 One CPU, 384 MiB memory and 64 PIDs per run.

02 Non-root process, read-only mounts and no network.

03 Default test deadline: 35 seconds. Go allows 75 seconds.

04 Container removal must complete.



Fresh verification results

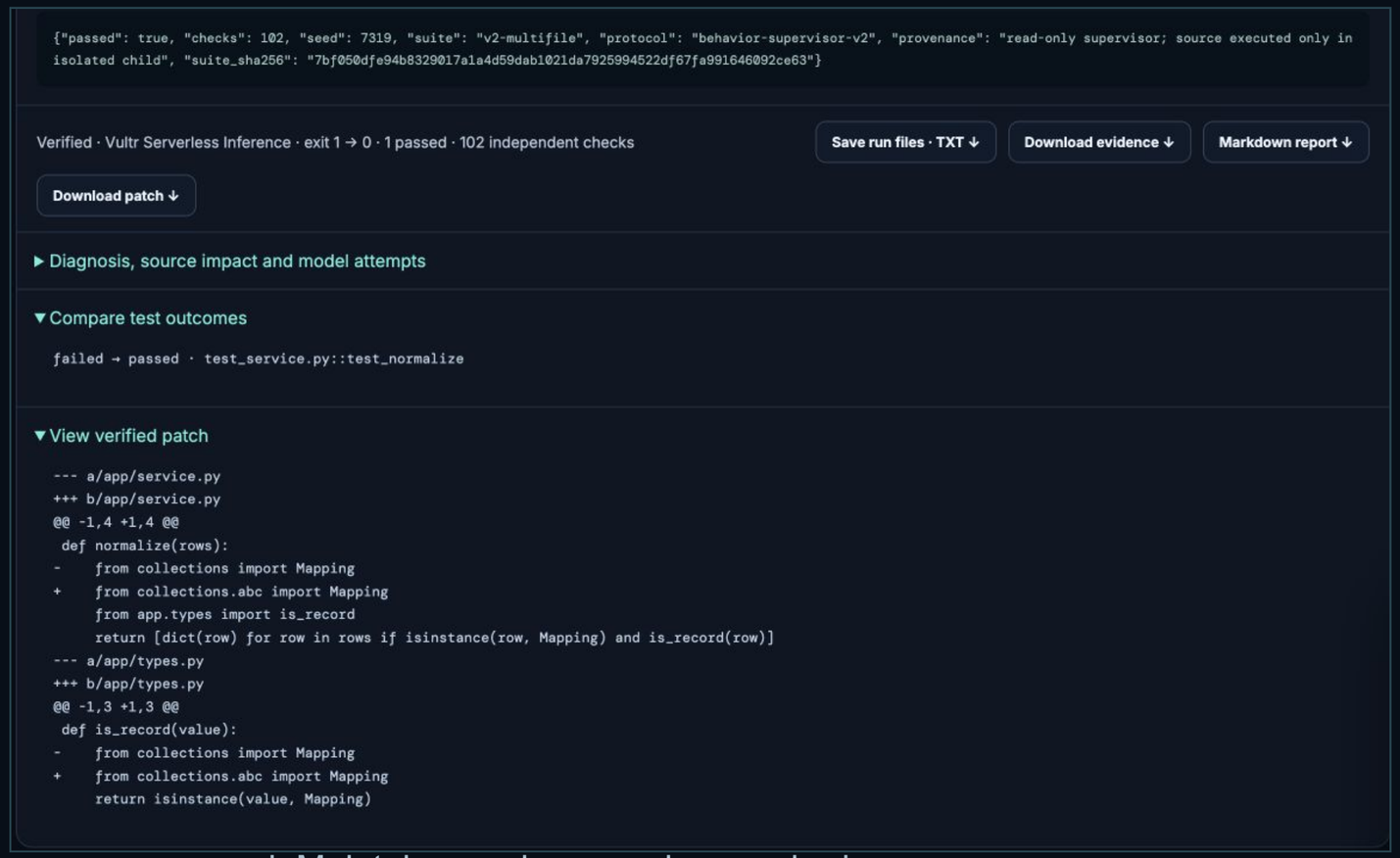
Run fe9d10292eb747ebb6d8ecd17cb61f6f

1

original test passed

102

independent checks passed



Both application files changed. All three execution containers were removed. Maintainer review remains required.

Diagnosis, source impact and models

The fresh run requested and served glm-5.3-flash through Vultr Serverless Inference.

Primary cloud model

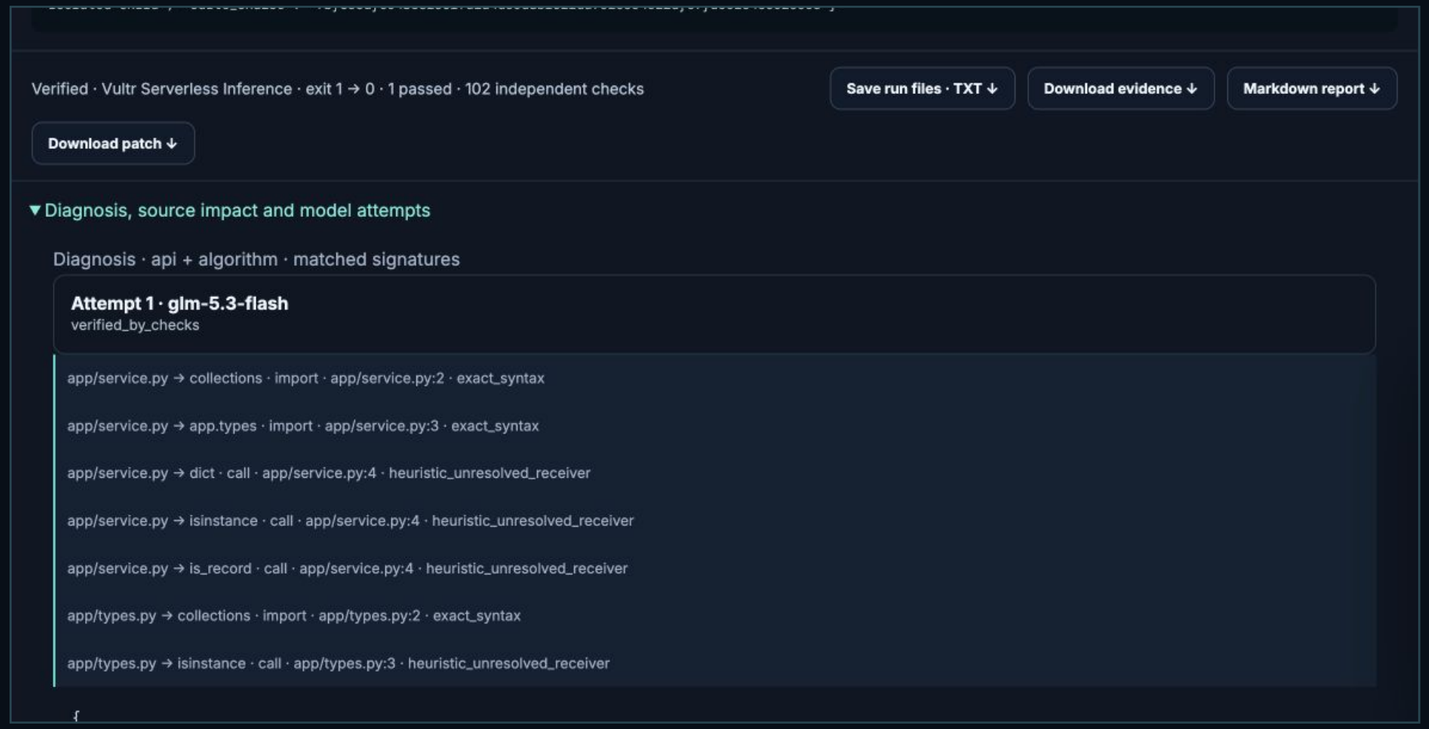
GLM 5.3 Flash

Bounded fallback

DeepSeek V4.1 Flash 1017

Optional private configuration

Qwen2.5-Coder 1.5B Q4_K_M on Dell via Ollama. Previously tested; not the backend used in this capture.



Downloadable evidence and handoff

Actual TXT export retrieved from this fresh run.

01 TXT: scoped files, output, changes and results.

02 JSON: recorded events and model receipts.

03 Markdown: a readable report.

04 Patch: available after successful verification.

```
VersionBridge AI - Run bundle
Outcome: Verified against listed checks
Run: fe9d10292eb747ebb6d8ecd17cb61f6f
Provider: Vultr Serverless Inference
Review: Maintainer decision required before applying a patch.
FILES / Original
Scoped application and test text snapshots; 16 files / 64 KiB per phase.
--- app/types.py ---
def is_record(value):
from collections import Mapping
return isinstance(value, Mapping)
```

Snapshot allowance: 16 files / 64 KiB per phase. The public workspace accepts reviewed sources and one exact pinned GitHub example.

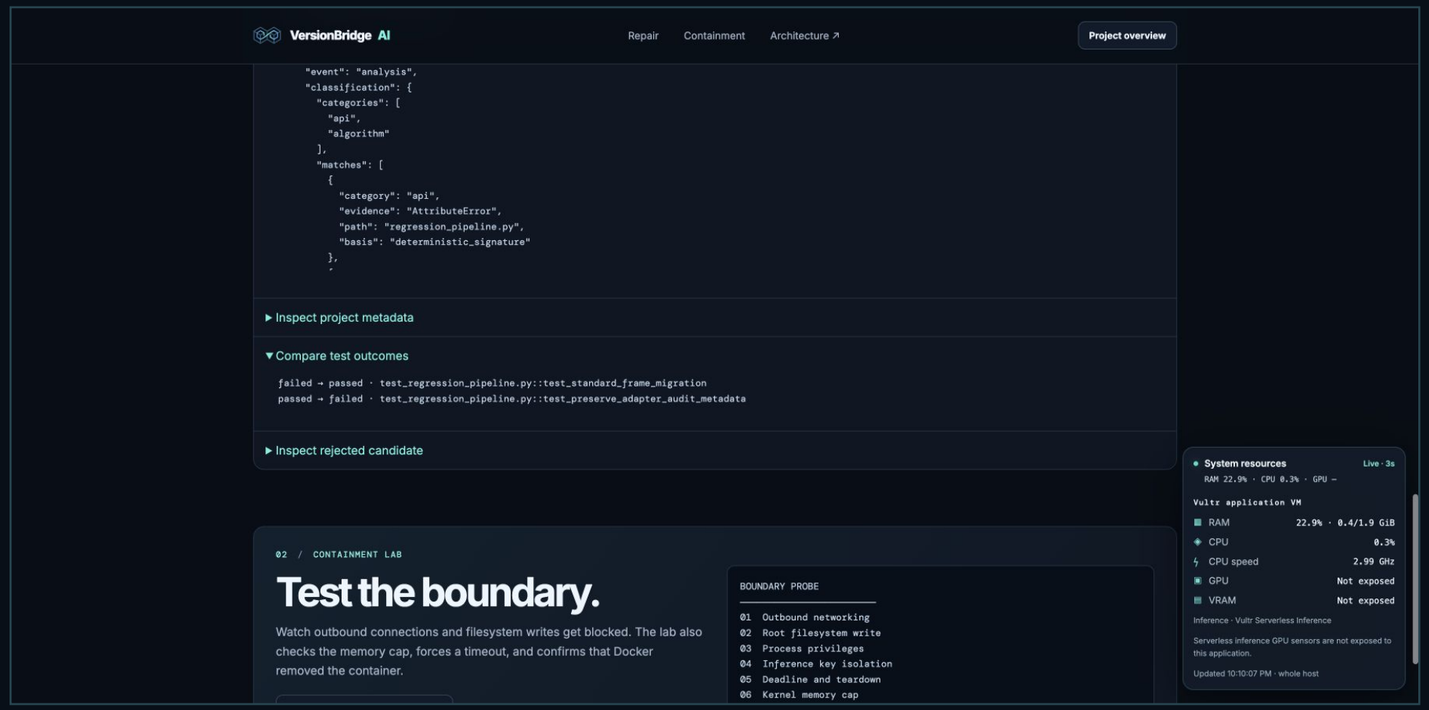
The regression guard rejects a harmful fix

Fresh negative run: the first failure changes, but the audit-metadata test fails.

01 The candidate repairs the removed pandas API call.

02 A second test catches lost audit information.

03 The candidate remains unverified. No verified patch is exported.



Containment checks passed

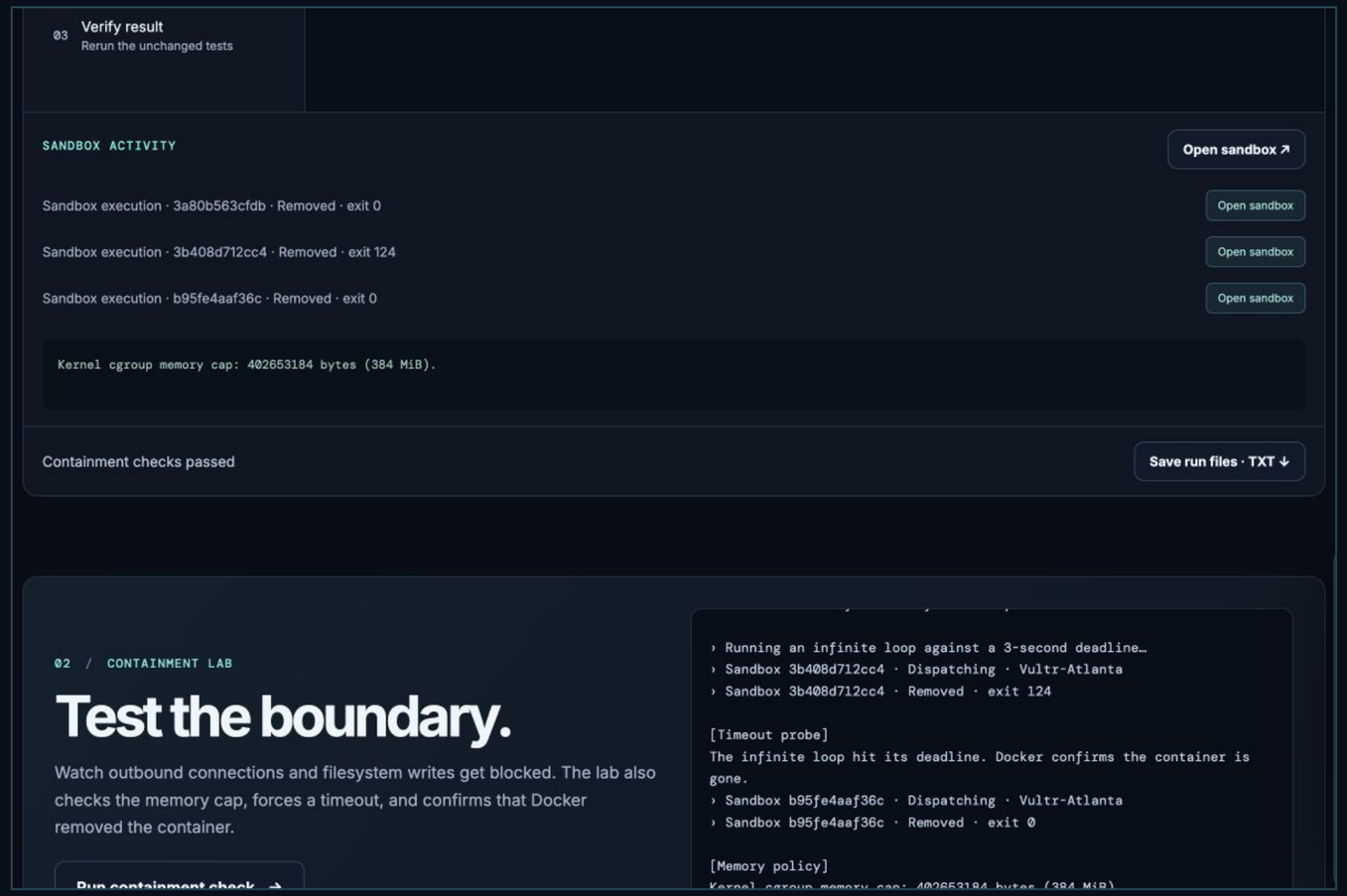
Fresh Containment Lab run. All three probe containers were removed.

01 Outbound network attempts and root writes fail.

02 Non-root user and absent inference key confirmed.

03 A loop hits its 3-second deadline; Docker removes it.

04 The kernel reports the 384 MiB memory cap.



Live system resources

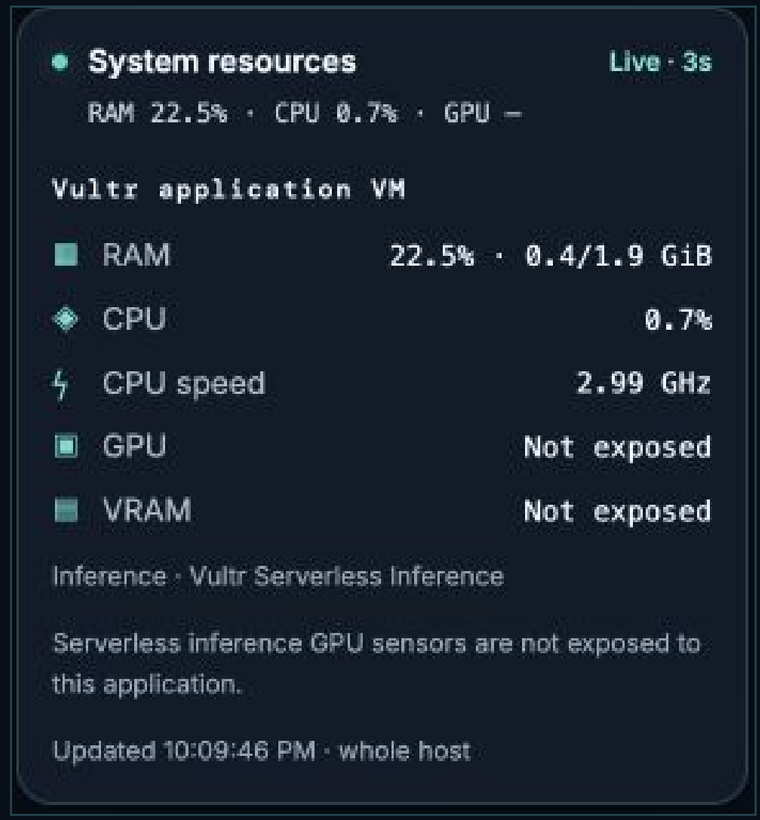
The panel refreshes whole-host readings and identifies their source.

01 RAM use, CPU use and available CPU speed.

02 GPU and VRAM when accessible local or trusted Dell sensors exist.

03 Serverless GPU sensors are not exposed; the UI says so.

04 Temperature is omitted from the visible panel.



Queue, recovery and operating controls

Implemented capabilities beyond the demonstrated repair.

Persistent jobs

Queued work survives restarts. Refresh recovery resumes job polling.

Cancellation

Stops at a step boundary. Recovery removes recorded containers before marking interruption.

Duplicate protection

Idempotency keys prevent duplicate submissions for the same request.

Access separation

Optional workspace bearer token. Separate web and worker OS users; only worker holds Docker access and model credentials.

Bounded usage

At most 5 active jobs; public limit 100 new jobs/day. Deployed inference allowance $4, 30 requests/day.

Retained evidence

Up to 100 completed jobs/runs. The removed history UI does not erase backend evidence.

Inference reservations are separate from VM, backup, bandwidth and other account charges.

Validation and fresh evidence

Release CI and this capture session measure different parts of the system.

56

Python test methods in the published release

27

HTTP integration checks in the published release

8

containers removed across these 3 fresh jobs

Fresh capture outcomes

Two-file migration	Verified: 1 original test and 102 independent checks passed.
Regression guard	Rejected by the unchanged tests; UI terminal state Unverified.
Containment Lab	Network, filesystem, privilege, key isolation, deadline, memory and cleanup checks passed.

Use cases and current boundaries

A working release for reviewed profiles, with a clearly defined next roadmap.

Useful today

01 Reviewed dependency and import migrations.

02 Selected code, algorithm and database-query repairs.

03 Teaching and auditing AI-generated changes through actual execution evidence.

Further development

01 General repository adapters and dependency preparation.

02 Private GitHub App access and automatic repair pull requests.

03 Organization permissions, generated test engines and stronger arbitrary-code isolation.

How to demonstrate VersionBridge

The evidence is ready to inspect in the public application and repository.

Suggested presentation order

01 Explain the problem and architecture.

02 Run the two-file repair and open the inspector.

03 Review the tests and patch, then save the TXT bundle.

04 Show the rejected regression and containment results.

Open the project

versionbridge.pages.dev

Source and documentation

github.com/Hari-Aditya-003/versionbridge-ai

Fresh run evidence

Repair report (JSON)

Containment report